

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



ANALAYSIS AND DESIGN OF ALGORITHMS AAT REPORT on

CODING CHALLENGES

Submitted by

Rohan S Mirjankar (1WN24CS232)

Rayeed Rizwan Sheikh (1WN24CS225)

Under the Guidance of

**Harsha M
Professor of Practice
BMSCE**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

February to June 2026

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, **Rohan S M (1WN24CS232)** and **Rayeed Rizwan Sheikh (1WN24CS225)**, students of 4th Semester U Section, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, hereby declare that, this Coding Challenges for the course Analysis and Design of Algorithms(23CS4PCADA) has been carried out by us under the guidance of **Harsha M**, Assistant Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester February to June 2026.

I also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

Rohan S M (1WN24CS232)

Rayeed Rizwan Sheikh (1WN24CS225)

Table of Contents

Sl no	Title	Page no
1	Coding challenge 3	1
2	Coding challenge 4	4
3	Coding challenge 5	7
4	Coding challenge 1	11
5	Coding challenge 2	14

Coding challenge 3:

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n . Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], $n = 2$

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], $n = 1$

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A","B","B","B"], $n = 3$

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

$1 \leq \text{tasks.length} \leq 104$

tasks[i] is an uppercase English letter.

$0 \leq n \leq 100$

Solution:

Design Technique chosen with justification:

Greedy Algorithm + Mathematical Counting

Justification

To minimize the total intervals, tasks with the highest frequency should be scheduled first.

The most frequent task determines the minimum number of slots needed.

Greedy counting of task frequencies allows direct computation without explicit simulation.

Algorithm:

```
int leastInterval(char* tasks, int tasksSize, int n) {
    int freq[26] = {0};

    for (int i = 0; i < tasksSize; i++)
        freq[tasks[i] - 'A']++;

    int maxFreq = 0;
    for (int i = 0; i < 26; i++) {
        if (freq[i] > maxFreq)
            maxFreq = freq[i];
    }

    int maxCount = 0;
    for (int i = 0; i < 26; i++) {
        if (freq[i] == maxFreq)
            maxCount++;
    }

    int result = (maxFreq - 1) * (n + 1) + maxCount;

    return (result > tasksSize) ? result : tasksSize;
}
```

Time Complexity:

We must:

Count frequency of each task (use array of size 26 or hash map) $\rightarrow O(m)$ where m = number of tasks

Find max_freq and max_count $\rightarrow O(26) = O(1)$ (since 26 letters only)

Compute formula $\rightarrow O(1)$

So:

Time Complexity = **$O(m)$** where m = length of tasks

Screenshots of the output from LeetCode platform:

The image displays two screenshots of the LeetCode platform interface, showing a user's profile and a problem solution.

Left Screenshot:

- User Profile:** rohan_s_m_14. Access all features with our Premium subscription!
- My Lists**, **Notebook**, **Progress**, **Points**
- Try New Features**, **Orders**, **My Playgrounds**, **Settings**, **Appearance**, **Sign Out**
- Code Editor:** C, Auto. `1 int leastInterval(char* tasks, in`
`2 | int freq[26] = {0};`
- Testcase** | **Test Result**
- Accepted** Runtime: 0 ms
- Case 1** **Case 2** **Case 3**
- Input:** tasks = ["A", "A", "A", "B", "B", "B"]
n = 2
- Output:** 8
- Expected:** 8
- Contribute a testcase**

Right Screenshot:

- Code Editor:** C, Auto. `1 int leastInterval(char* tasks, in`
`2 | int freq[26] = {0};`
- Testcase** | **Test Result**
- Accepted** Runtime: 0 ms
- Case 1** **Case 2** **Case 3**
- Input:** tasks = ["A", "C", "A", "B", "D", "B"]
n = 1
- Output:** 6
- Expected:** 6

Coding challenge 4:

Given an array of integers nums, sort the array in ascending order and return it.

You must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity and with the smallest space complexity possible.

Example 1:

Input: nums = [5,2,3,1]

Output: [1,2,3,5]

Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).

Example 2:

Input: nums = [5,1,1,2,0,0]

Output: [0,0,1,1,2,5]

Explanation: Note that the values of nums are not necessarily unique.

Constraints:

$1 \leq \text{nums.length} \leq 5 * 10^4$

$-5 * 10^4 \leq \text{nums}[i] \leq 5 * 10^4$

Solution:

Design Technique chosen with justification:

Divide and Conquer (Merge Sort)

Justification

Merge Sort divides the array into smaller subarrays recursively.

Each subarray is sorted independently.

Sorted subarrays are merged to obtain the final sorted array.

It guarantees $O(n \log n)$ time complexity in all cases, making it suitable for large inputs.

Algorithm:

```
void merge(int* nums, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int* L = (int*)malloc(n1 * sizeof(int));
    int* R = (int*)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++)
        L[i] = nums[left + i];

    for (int j = 0; j < n2; j++)
```

```

        R[j] = nums[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            nums[k++] = L[i++];
        else
            nums[k++] = R[j++];
    }

    while (i < n1)
        nums[k++] = L[i++];

    while (j < n2)
        nums[k++] = R[j++];

    free(L);
    free(R);
}

void mergeSort(int* nums, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(nums, left, mid);
        mergeSort(nums, mid + 1, right);

        merge(nums, left, mid, right);
    }
}

int* sortArray(int* nums, int numsSize, int* returnSize) {
    mergeSort(nums, 0, numsSize - 1);

    *returnSize = numsSize;
    return nums;
}

```

Time Complexity:

Let $T(n)$ = time to sort n elements
 Recurrence relation: $T(n) = 2T(n/2) + O(n)$

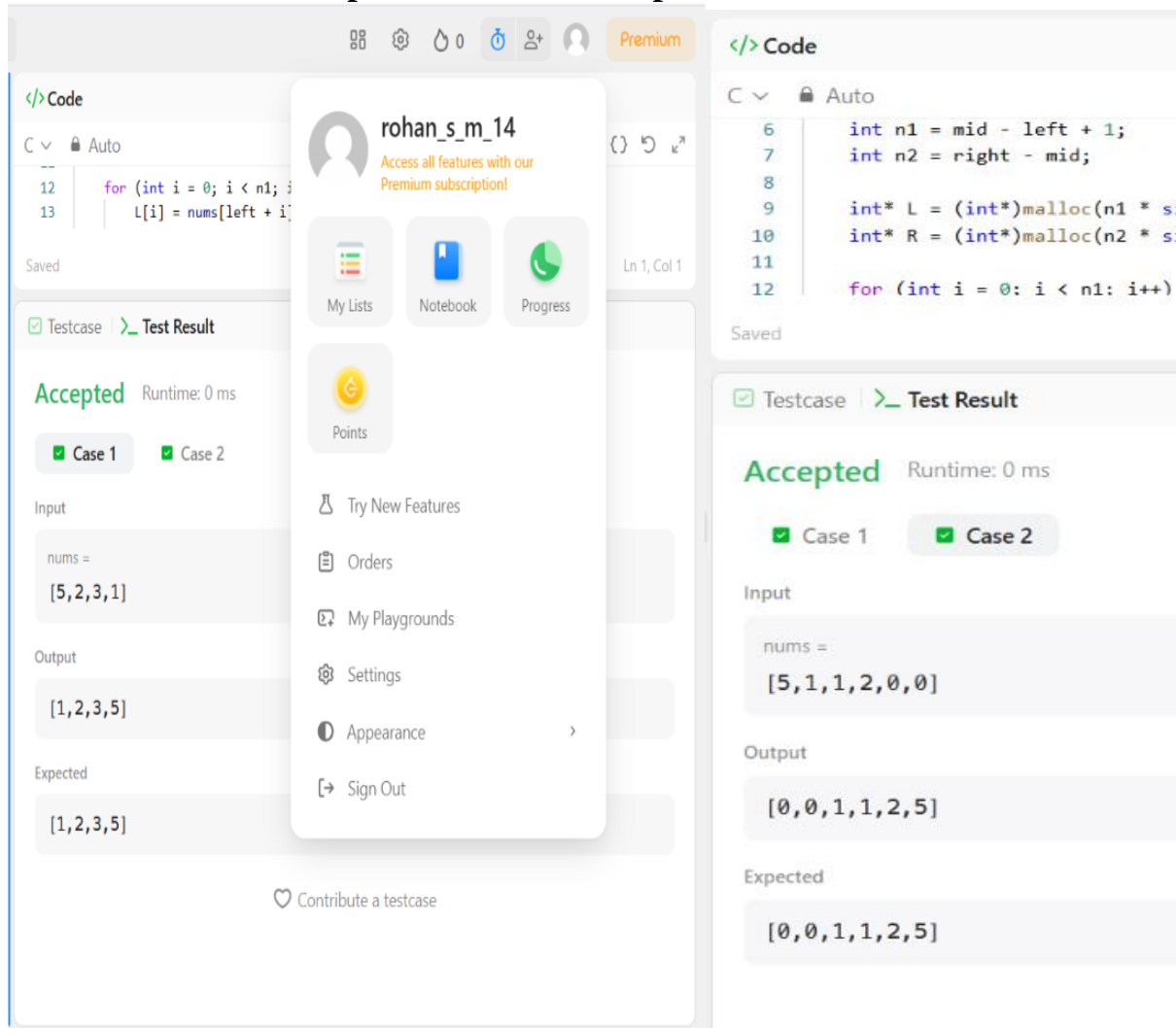
$T(n) = 2T(n/2) + n$
 $T(n/2) = 2T(n/4) + n/2$
 $T(n/4) = 2T(n/8) + n/4$
 ... continue until $T(1) = O(1)$

Total: $T(n) = n + n + n + \dots (\log n \text{ times}) + T(1)$

$T(n) = n \log n + O(1)$

Thus $O(n \log n)$

Screenshots of the output from LeetCode platform:



Coding challenge 5:

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$). You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:

Input: $\text{points} = [[1,3],[-2,2]]$, $k = 1$

Output: $[-2,2]$

Explanation:

The distance between $(1, 3)$ and the origin is $\sqrt{10}$.

The distance between $(-2, 2)$ and the origin is $\sqrt{8}$.

Since $\sqrt{8} < \sqrt{10}$, $(-2, 2)$ is closer to the origin.

We only want the closest $k = 1$ points from the origin, so the answer is just $[-2,2]$.

Example 2:

Input: $\text{points} = [[3,3],[5,-1],[-2,4]]$, $k = 2$

Output: $[[3,3],[-2,4]]$

Explanation: The answer $[-2,4],[3,3]$ would also be accepted.

Constraints:

$1 \leq k \leq \text{points.length} \leq 104$

$-104 \leq x_i, y_i \leq 104$

Solution:

Design Technique chosen with justification:

Heap (Priority Queue) / Max Heap

Justification

We need only the k closest points, not a complete sorting of all points.

Maintain a max heap of size k .

If a new point is closer than the farthest point in the heap, replace it.

This efficiently keeps track of the k closest points.

Algorithm:

```
typedef struct {
    int dist;
    int index;
} Node;

void swap(Node* a, Node* b) {
    Node temp = *a;
    *a = *b;
```

```

    *b = temp;
}

void heapify(Node heap[], int size, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < size && heap[left].dist > heap[largest].dist)
        largest = left;

    if (right < size && heap[right].dist > heap[largest].dist)
        largest = right;

    if (largest != i) {
        swap(&heap[i], &heap[largest]);
        heapify(heap, size, largest);
    }
}

int** kClosest(int** points, int pointsSize, int* pointsColSize,
               int k, int* returnSize, int** returnColumnSizes) {

    Node* heap = (Node*)malloc(k * sizeof(Node));
    int heapSize = 0;

    for (int i = 0; i < pointsSize; i++) {

        int x = points[i][0];
        int y = points[i][1];
        int dist = x * x + y * y;

        if (heapSize < k) {

            heap[heapSize].dist = dist;
            heap[heapSize].index = i;
            int cur = heapSize++;

            while (cur > 0) {
                int parent = (cur - 1) / 2;

                if (heap[parent].dist >= heap[cur].dist)
                    break;

                swap(&heap[parent], &heap[cur]);
                cur = parent;
            }
        }
        else if (dist < heap[0].dist) {

            heap[0].dist = dist;

```

```

        heap[0].index = i;

        heapify(heap, k, 0);
    }
}

int** result = (int**)malloc(k * sizeof(int*));

*returnColumnSizes = (int*)malloc(k * sizeof(int));

for (int i = 0; i < k; i++) {
    result[i] = (int*)malloc(2 * sizeof(int));

    result[i][0] = points[heap[i].index][0];
    result[i][1] = points[heap[i].index][1];

    (*returnColumnSizes)[i] = 2;
}

*returnSize = k;

free(heap);

return result;
}

```

Time Complexity:

Step 1 — Partition cost
Partitioning takes $O(n)$ time:

Choose a pivot

Rearrange points so smaller distances are on left, larger on right

Step 2 — Recursion
After partition:

Pivot ends at index p

If $p == k-1$, we're done (found k -th smallest)

If $p > k-1$, recurse on left side (indices $0..p-1$)

If $p < k-1$, recurse on right side (indices $p+1..n-1$), but we want $(k-p-1)$ -th smallest there

Step 3 — Recurrence relation
Unlike quicksort which recurses on both sides, Quick Select recurses on only one side.

Let $T(n)$ = time for finding k -th smallest in array of size n

Recurrence: $T(n) = T(n/2) + O(n)$

Step 4 — Solve recurrence

$T(n) = T(n/2) + n$

$T(n/2) = T(n/4) + n/2$

$T(n/4) = T(n/8) + n/4$

...

$T(1) = O(1)$

Step 5 — Worst case

Worst case (always bad pivot) = $O(n^2)$

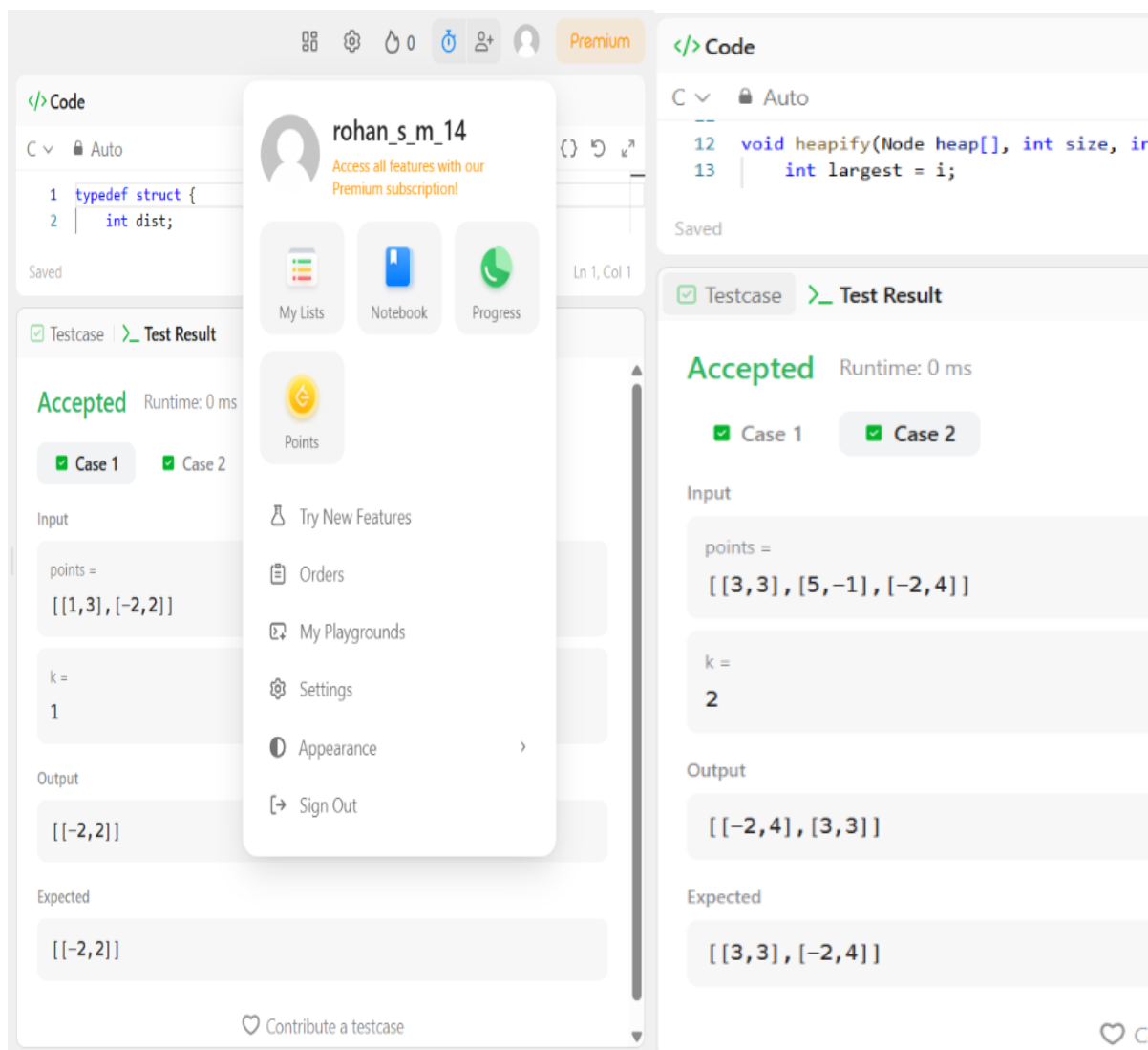
But we can avoid by:

Random pivot selection

Median-of-medians (guarantees $O(n)$, but complex)

Average case (practical) = $O(n)$

Screenshots of the output from LeetCode platform:



Coding challenge 1:

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

$n == \text{height.length}$

$1 \leq n \leq 2 * 10^4$

$0 \leq \text{height}[i] \leq 10^5$

Solution:

Design Technique chosen with justification:

Two Pointer Technique (Greedy + Space Optimization)

Justification:

Water trapped at an index depends on the minimum of the maximum heights on its left and right.

Instead of precomputing leftMax[] and rightMax[] arrays, two pointers can track these values dynamically.

This reduces extra space from $O(n)$ to $O(1)$.

Algorithm:

```
int trap(int* height, int heightSize) {
    int left = 0, right = heightSize - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;

    while (left < right) {
```

```

    if (height[left] < height[right]) {
        if (height[left] >= leftMax)
            leftMax = height[left];
        else
            water += leftMax - height[left];
        left++;
    } else {
        if (height[right] >= rightMax)
            rightMax = height[right];
        else
            water += rightMax - height[right];
        right--;
    }
}

return water;
}

```

Time Complexity:

Loop runs until $\text{left} < \text{right}$:

n = length of array.

At iteration k ($1..n-1$ maybe), we compare $\text{height}[\text{left}]$ and $\text{height}[\text{right}]$.

Each iteration does a constant amount of work and advances one pointer by 1.

Total number of iterations = initially $\text{right} - \text{left} = n-1$.

In each iteration, $\text{right} - \text{left}$ decreases by exactly 1.

Loop ends when $\text{right} - \text{left}$ goes from $n-1$ to 0.

Number of iterations = $n-1$.

Thus, **$O(n)$** .

Screenshots of the output from LeetCode platform:

The image displays two screenshots of the LeetCode platform interface. The left screenshot shows a user's profile for 'rayeed_rizwan' with a 'Premium' subscription. The profile card includes a 'Points' section with a gold coin icon, and a list of features: 'Try New Features', 'Orders', 'My Playgrounds', 'Settings', 'Appearance', and 'Sign Out'. Below the profile card, there is a 'Contribute a testcase' button. The background shows a code editor with a C++ solution for the 'trap' problem, which has been accepted. The input is '[0,1,0,2,1,0,1,3,2,1,2,1]' and the output is '6'. The right screenshot shows the same code editor with a different input: '[4,2,0,3,2,5]'. The output is '9'. The code is the same C++ solution for the 'trap' problem.

Left Screenshot:

- User Profile:** rayeed_rizwan (Premium subscription)
- Points:** 0
- Features:** Try New Features, Orders, My Playgrounds, Settings, Appearance, Sign Out
- Testcase Result:** Accepted (Runtime: 0 ms)
- Case 1:** [0,1,0,2,1,0,1,3,2,1,2,1] → 6
- Case 2:** [0,1,0,2,1,0,1,3,2,1,2,1] → 6

Right Screenshot:

- Code:** C++ solution for 'trap' problem
- Testcase Result:** Accepted (Runtime: 0 ms)
- Case 1:** [4,2,0,3,2,5] → 9
- Case 2:** [4,2,0,3,2,5] → 9

Coding challenge 2:

You are given an array `books` where `books[i] = [thicknessi, heighti]` indicates the thickness and height of the *i*th book. You are also given an integer `shelfWidth`. We want to place these books in order onto bookcase shelves that have a total width `shelfWidth`. We choose some of the books to place on this shelf such that the sum of their thickness is less than or equal to `shelfWidth`, then build another level of the shelf of the bookcase so that the total height of the bookcase has increased by the maximum height of the books we just put down. We repeat this process until there are no more books to place.

Note that at each step of the above process, the order of the books we place is the same order as the given sequence of books.

For example, if we have an ordered list of 5 books, we might place the first and second book onto the first shelf, the third book on the second shelf, and the fourth and fifth book on the last shelf. Return the minimum possible height that the total bookshelf can be after placing shelves in this manner.

Example 1:

Input: `books = [[1,1],[2,3],[2,3],[1,1],[1,1],[1,1],[1,2]]`, `shelfWidth = 4`

Output: 6

Explanation:

The sum of the heights of the 3 shelves is $1 + 3 + 2 = 6$.

Notice that book number 2 does not have to be on the first shelf.

Example 2:

Input: `books = [[1,3],[2,4],[3,2]]`, `shelfWidth = 6`

Output: 4

Constraints:

$1 \leq \text{books.length} \leq 1000$

$1 \leq \text{thicknessi} \leq \text{shelfWidth} \leq 1000$

$1 \leq \text{heighti} \leq 1000$

Solution:

Design Technique chosen with justification:

Dynamic Programming (DP)

Justification

The minimum height of the bookshelf depends on how books are grouped on shelves. For each book, we try placing previous books on the same shelf (if width permits) and compute the minimum total height.

Since optimal solutions of smaller subproblems are reused, Dynamic Programming is appropriate.

Algorithm:

```
int minHeightShelves(int** books, int booksSize, int* booksColSize, int shelfWidth) {
    int dp[1001];

    dp[0] = 0;

    for (int i = 1; i <= booksSize; i++) {
        dp[i] = 1000000;

        int width = 0;
        int height = 0;

        for (int j = i; j >= 1; j--) {
            width += books[j - 1][0];

            if (width > shelfWidth)
                break;

            if (books[j - 1][1] > height)
                height = books[j - 1][1];

            if (dp[j - 1] + height < dp[i])
                dp[i] = dp[j - 1] + height;
        }
    }

    return dp[booksSize];
}
```

Time Complexity:

We iterate i from 1 to n.

For each i, we try all j from i-1 down to 0, accumulating total width and max height until total width exceeds shelf_width.

Inner loop:

Starts at j = i-1

Moves left to $j = 0$ or until $\text{width} > \text{shelf_width}$

Each iteration (for a given i) does $O(1)$ work per j (update max, width check)

Number of Operations

In worst case, width constraint is large (e.g., $\text{shelf_width} > \text{total thickness}$), so every $j < i$ is considered.

For $i = 1$, inner loop runs 1 time ($j=0$)

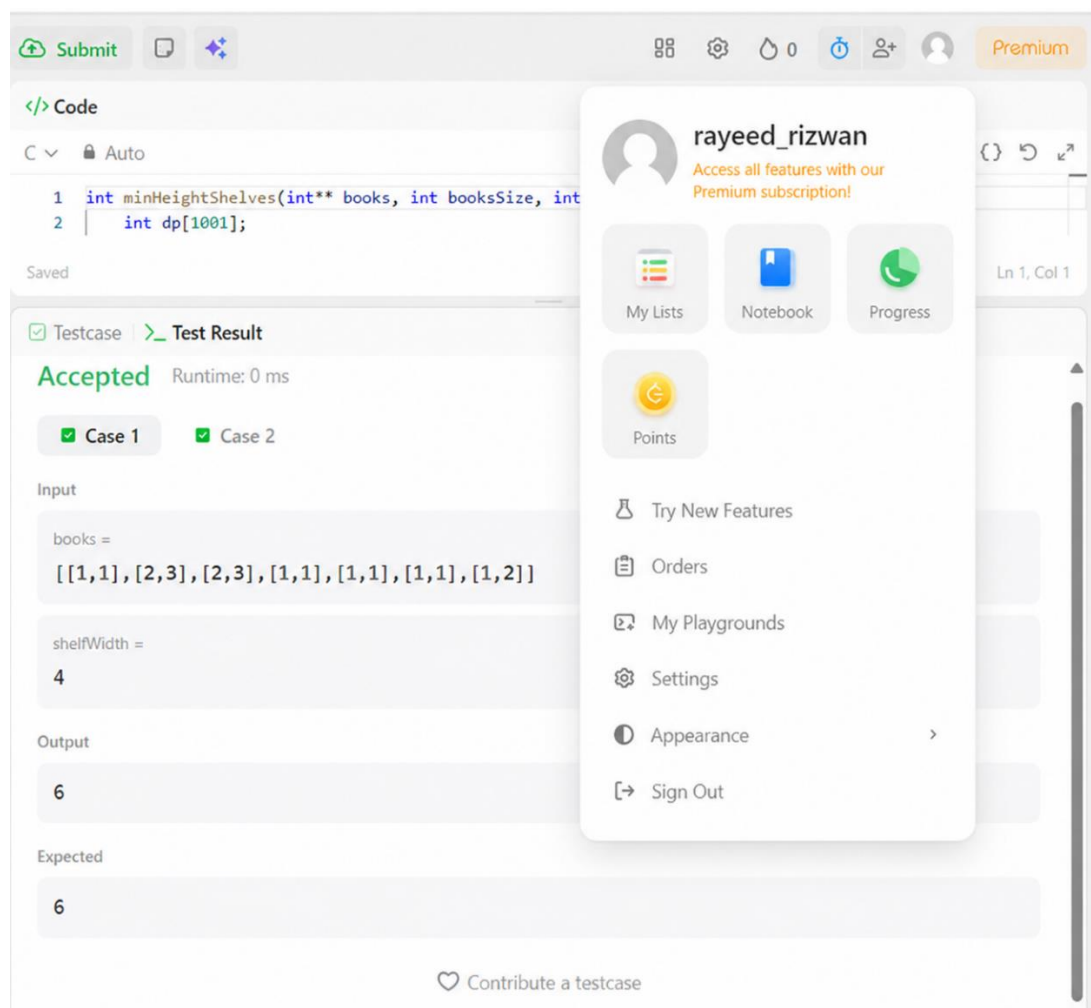
$i = 2$, inner loop runs 2 times ($j=0,1$)

$i = n$, inner loop runs n times ($j=0..n-1$) (if no early stop)

Total inner loop iterations for all $i = 1 + 2 + 3 + \dots + n = O(n^2)$

So time complexity = $O(n^2)$.

Screenshots of the output from LeetCode platform:



</> Code

C ▾ 🔒 Auto

☰ 📖 {} ↺ ↻ ↶ ↷

```
1 int minHeightShelves(int** books, int booksSize, int* booksColSize, int shelfWidth) {
2     int dp[1001];
```

SavedLn 27, Col 2

☑ Testcase >_ Test Result

Accepted Runtime: 0 ms

☑ Case 1

☑ Case 2

Input

books =
[[1,3],[2,4],[3,2]]

shelfWidth =
6

Output

4

Expected

4

♥ Contribute a testcase